

Teoría de la Computación, 2021-1

Nota de clase 6: Gramáticas Libres de Contexto

Manuel Soto Romero

19 de mayo de 2021
Colegio de Ciencia y Tecnología UACM

6.1. Introducción

Como hemos visto, un mecanismo relevante para generar un lenguaje es mediante el concepto de gramática formal. Las gramáticas formales fueron introducidas por Noam Chomsky en 1956. La intención era tener un modelo para la descripción de lenguajes naturales. Posteriormente una clase especial de estos formalismos, llamados gramáticas libres de contexto, se utilizaron como herramienta para presentar la sintaxis de lenguajes de programación y para el diseño de analizadores léxicos de compiladores.

Definición 6.1. (GLC) *Una gramática libre de contexto (GLC) es una cuaterna $G = \langle V, T, S, P \rangle$ tal que:*

- *V es un alfabeto de variables o símbolos no terminales, los cuales se denotan por mayúsculas $A, B, C, \dots$*
- *T es un alfabeto de constantes símbolos terminales, los cuales se denotan por minúsculas a, b, c con la condición de que $T \cap V = \emptyset$.*
- *$S \in V$ es una variable distinguida llamada el símbolo inicial.*
- *P es un conjunto finito de reglas de producción, donde cada regla es de la forma*

$$A \rightarrow \alpha$$

y $A \in V, \alpha \in (V \cup T)^*$.

Veamos algunos ejemplos:

- Gramática que reconoce el lenguaje a^* :

$$\begin{aligned} S &\rightarrow aS \\ S &\rightarrow \varepsilon \end{aligned}$$

- Gramática que reconoce al lenguaje a^*b^* :

$$\begin{aligned} S &\rightarrow aS \\ S &\rightarrow bA \\ S &\rightarrow \varepsilon \\ A &\rightarrow bA \\ A &\rightarrow b \\ A &\rightarrow \varepsilon \end{aligned}$$

- Gramática que reconoce al lenguaje $0^*10^*10^*$:

$$\begin{aligned} S &\rightarrow A1A1A \\ A &\rightarrow 0A \\ A &\rightarrow \varepsilon \end{aligned}$$

- Gramática que genere el lenguaje $L = \{a^i b^i : i \geq 0\}$:

$$\begin{aligned} S &\rightarrow aSb \\ S &\rightarrow \varepsilon \end{aligned}$$

Observación 6.1. Es importante notar que toda gramática regular es libre de contexto pero no al revés.

Ejercicio 6.1. Encontrar una GLC que genere el lenguaje de todos los palíndromos sobre $\Sigma = \{a, b\}$.

Ejercicio 6.2. Encontrar una GLC que genere el lenguaje de todas las cadenas sobre $\Sigma = \{a, b\}$ que tienen un número par de símbolos.

6.2. Gramáticas ambiguas

Recordemos que dada una gramática podemos dar una derivación que permita analizar la forma en que una cadena puede ser construida así como analizar su respectivo árbol de derivación. En este caso, una derivación usando gramáticas libres de contexto no suele ser única pues cada paso depende de la variable elegida para su reescritura. Esto genera un no determinismo que puede eliminarse usando una convención, la cual suele ser de dos tipos, eligiendo siempre la variable más a la izquierda o bien más a la derecha para el proceso de reescritura.

Definición 6.2. (*Derivación por la izquierda*) Una derivación es por la izquierda si en cada paso se reescribe la variable más a la izquierda en la cadena.

Definición 6.3. (*Derivación por la derecha*) Una derivación es por la derecha si en cada paso se reescribe la variable más a la derecha en la cadena.

Ejemplo 6.1. En la gramática

$$\begin{aligned} S &\rightarrow aSb \\ S &\rightarrow a \\ A &\rightarrow SbA \\ A &\rightarrow SS \\ A &\rightarrow ba \end{aligned}$$

1. La siguiente derivación por la izquierda de la cadena $aabbaa$:

$$S \rightarrow aAS \rightarrow aSbAS \rightarrow aabAS \rightarrow aabbaS \rightarrow aabbaa$$

2. La siguiente derivación por la derecha de la cadena $aabbaa$:

$$S \rightarrow aAS \rightarrow aAa \rightarrow aSbAa \rightarrow aSbbaa \rightarrow aabbaa$$

Definición 6.4. (Gramática ambigua) Una GLC es ambigua si existe una cadena para la cual existen dos derivaciones diferentes. De la misma forma, podemos decir que es ambigua si existen dos árboles de derivación diferentes para la misma cadena.

Ejemplo 6.2. Consideremos el alfabeto $\Sigma = \{0, 1, +, *, (,)\}$. La siguiente gramática para generar número naturales, sumas y productos, en numeración binaria es ambigua, recordemos que el símbolo $|$ se lee como “o” y permite simplificar las reglas de escritura:

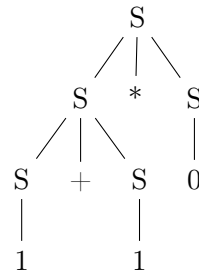
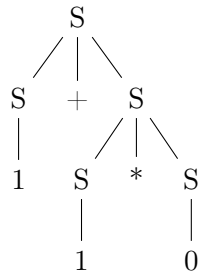
$$S \rightarrow S + S \mid S * S \mid (S) \mid 0S \mid 1S \mid 0S \mid 1S \mid 0 \mid 1$$

La cadena $1 + 1 * 0$ tiene dos derivaciones por la izquierda diferentes:

$$S \rightarrow S + S \rightarrow 1 + S \rightarrow 1 + S * S \rightarrow 1 + 1 * S \rightarrow 1 + 1 * 0$$

$$S \rightarrow S * S \rightarrow S + S * S \rightarrow 1 + S * S \rightarrow 1 + 1 * S \rightarrow 1 + 1 * 0$$

Los árboles de derivación correspondientes a la derivaciones anteriores son:



La ambigüedad se puede eliminar introduciendo paréntesis:

$$S \rightarrow (S + S) \mid (S * S) \mid (S) \mid 0S \mid 1S \mid 0S \mid 1S \mid 0 \mid 1$$

Sin embargo, aunque la adición de los paréntesis elimina la ambigüedad, las expresiones generadas tienen un excesivo número de paréntesis lo que dificulta el análisis sintáctico (en un compilador por ejemplo). Lo más común en estos casos es utilizar gramáticas como la original siempre y cuando se establezca un orden de precedencia para los operadores. Lo usual es establecer que $*$ tenga una mayor precedencia que $+$. Por ejemplo, la expresión $1*1+0$ se interpreta como $(1*1)+0$ y la expresión $10+11*110+1$ se interpreta como $10+(11*110)+1$.

Ejercicio 6.3. Indica si la siguiente gramática es ambigua. Justifica tu respuesta.

$$\begin{aligned} S &\rightarrow aSa \mid \varepsilon \\ A &\rightarrow bA \mid \varepsilon \end{aligned}$$

6.3. Normalización de gramáticas

La normalización de gramáticas consiste en transformar todas las producciones de una gramática de manera que tengan cierta forma sintáctica en particular. Así la normalización de gramáticas libres de contexto es útil para homogeneizar la forma de las producciones, además para optimizar los procesos de derivación de cadenas. Con las *Formas Normales*, se facilita una solución al problema de la pertenencia, es decir, decidir si una cadena pertenece o no a un lenguaje:

Dada una gramática G y una cadena w , ¿se cumple que $w \in L(G)$?

- Si la palabra w es generada por G , el proceso para la construcción de un árbol de derivación terminará eventualmente.
- En caso contrario **no** podemos saber cuando parar en la construcción de un árbol de derivación.
- Pero si la gramática está en cierta forma normal, sí es posible saber cuando parar en la búsqueda de árboles de derivación para contestar el problema de la pertenencia.

Para obtener gramáticas normales, son necesarias varias transformaciones que simplificarán y reorganizarán las producciones.

6.3.1. Eliminación de variables inútiles

En una GLC puede haber dos tipos de variables inútiles: aquéllas que nunca aparecen en el proceso de ninguna derivación y aquéllas que no se pueden convertir en cadenas terminales.

Definición 6.5. (*Variable alcanzable*) Una variable A es alcanzable si existen $u, v \in (V \cup \Sigma)^*$ tales que $S \Rightarrow^* uAv$. La variable inicial S es alcanzable por definición.

Definición 6.6. (*Variable terminal*) Una variable A es terminal si existen $w \in \Sigma^*$ tales que $A \Rightarrow^* w$. En particular si $A \rightarrow \varepsilon$ es una producción, entonces A es terminal.

Algoritmo para encontrar las variables terminales

El siguiente algoritmo sirve para encontrar todas las variables terminales de una GLC.

TERM₁ := $\{A \in V : \text{Existe una producción de la forma } A \rightarrow w, w \in \Sigma^*\}$.

TERM_{i+1} := **TERM_i** $\cup \{A \in V : \exists \text{ producción } A \rightarrow w, w \in (\Sigma \cup \text{TERM}_i)^*\}$.

De esta forma obtenemos una sucesión creciente de conjuntos de variables:

$$TERM_1 \subseteq TERM_2 \subseteq TERM_3 \subseteq \dots$$

El algoritmo se presenta entonces como:

- Inicializar: **TERM** := $\{A \in V : \text{Existe una producción de la forma } A \rightarrow w, w \in \Sigma^*\}$.
- Repetir: **TERM** := **TERM** $\cup \{A \in V : \exists \text{ producción } A \rightarrow w, w \in (\Sigma \cup TERM)^*\}$.
- Hasta: No se añaden nuevas variables a **TERM**.

Ejemplo 6.3. Encontrar las variables terminales de la siguiente gramática

$$\begin{aligned} S &\rightarrow ACD \mid bBd \mid ab \\ A &\rightarrow aB \mid aA \mid C \\ B &\rightarrow aDS \mid aB \\ C &\rightarrow aCS \mid CB \mid CC \\ D &\rightarrow bD \mid ba \\ E &\rightarrow AB \mid aDb \end{aligned}$$

Solución

$$TERM_1 = \{S, D\}$$

$$TERM_2 = \{S, D\} \cup \{B, E\} = \{S, D, B, E\}$$

$$TERM_3 = \{S, D, B, E\} \cup \{A\} = \{S, D, B, E, A\}$$

$$TERM_4 = \{S, D, B, E, A\} \cup \emptyset = \{S, D, B, E, A\}$$

$\therefore$ El conjunto de variables no terminales es $\{C\}$

Algoritmo para encontrar las variables alcanzables

El siguiente algoritmo sirve para encontrar todas las variables alcanzables de una GLC.

$$ALC_1 := \{S\}.$$

$$ALC_{i+1} := ALC_i \cup \{A \in V : \exists \text{ producción } B \rightarrow uAv, B \in ALC_i, u, v \in (V \cup \Sigma)^*\}.$$

De esta forma obtenemos una sucesión creciente de conjuntos de variables:

$$ALC_1 \subseteq ALC_2 \subseteq ALC_3 \subseteq \dots$$

El algoritmo se presenta entonces como:

- Inicializar: **ALC** := $\{S\}$.
- Repetir: **ALC** := **ALC** $\cup \{A \in V : \exists \text{ producción } B \rightarrow uAv, B \in ALC, u, v \in (V \cup \Sigma)^*\}$.

- Hasta: No se añaden nuevas variables a **ALC**.

Ejemplo 6.4. Encontrar las variables terminales de la siguiente gramática

$$\begin{aligned}
S &\rightarrow aS \mid AaB \mid ACS \\
A &\rightarrow aS \mid AaB \mid AC \\
B &\rightarrow bB \mid DB \mid BB \\
C &\rightarrow aDa \mid ABD \mid ab \\
D &\rightarrow aD \mid DD \mid ab \\
E &\rightarrow FF \mid aa \\
F &\rightarrow aE \mid EF
\end{aligned}$$

Solución

$$\begin{aligned}
\mathbf{ALC}_1 &= \{S\} \\
\mathbf{ALC}_2 &= \{S\} \cup \{A, B, C\} = \{S, A, B, C\} \\
\mathbf{ALC}_3 &= \{S, A, B, C\} \cup \{D\} = \{S, A, B, C, D\} \\
\mathbf{ALC}_4 &= \{S, A, B, C, D\} \cup \emptyset = \{S, A, B, C, D\}
\end{aligned}$$

$\therefore$ El conjunto de variables no alcanzables es $\{E, F\}$

6.3.2. Eliminación de producciones ε

Definición 6.7. (*Producción*) Una producción de la forma $A \rightarrow \varepsilon$ se llama *producción ε*

Definición 6.8. (*Variable anulable*) Una variable A se llama *anulable* si $A \Rightarrow^* \varepsilon$.

Algoritmo para encontrar las variables anulables

El siguiente algoritmo sirve para encontrar todas las variables alcanzables de una GLC.

$$\begin{aligned}
\mathbf{ANUL}_1 &:= \{A \in V : A \rightarrow \varepsilon \text{ es una producción}\}. \\
\mathbf{ANUL}_{i+1} &:= \mathbf{ANUL}_i \cup \{A \in V : \exists \text{ producción } A \rightarrow w, w \in (\mathbf{ANUL}_i)^*\}.
\end{aligned}$$

De esta forma obtenemos una sucesión creciente de conjuntos de variables:

$$\mathbf{ANUL}_1 \subseteq \mathbf{ANUL}_2 \subseteq \mathbf{ANUL}_3 \subseteq \dots$$

El algoritmo se presenta entonces como:

- Inicializar: $\mathbf{ANUL} := \{A \in V : A \rightarrow \varepsilon \text{ es una producción}\}.$
- Repetir: $\mathbf{ANUL} := \mathbf{ANUL} \cup \{A \in V : \exists \text{ producción } A \rightarrow w, w \in (\mathbf{ANUL})^*\}.$

- Hasta: No se añaden nuevas variables a **ANUL**.

Una vez que haya sido determinado el conjunto **ANUL** de variables anulables, por medio del algoritmo anterior, las producciones ε se pueden eliminar (excepto $S \rightarrow \varepsilon$) añadiendo nuevas producciones que simulen el efecto de las producciones ε eliminadas. Más concretamente, por cada producción $A \rightarrow u$ se añaden producciones de la forma $A \rightarrow v$ obtenidas suprimiendo de la cadena u una, dos o más variable anulables presentes de todas las formas posibles.

Ejemplo 6.5. Encontrar las producciones ε de la siguiente gramática

$$\begin{aligned} S &\rightarrow AB \mid ACA \mid ab \\ A &\rightarrow aAa \mid B \mid CD \\ B &\rightarrow bB \mid bA \\ C &\rightarrow cC \mid \varepsilon \\ D &\rightarrow aDc \mid CC \mid ABb \end{aligned}$$

Solución

$$\begin{aligned} \text{ANUL}_1 &= \{C\} \\ \text{ANUL}_2 &= \{C\} \cup \{D\} = \{C, D\} \\ \text{ANUL}_3 &= \{C, D\} \cup \{A\} = \{C, D, A\} \\ \text{ANUL}_4 &= \{C, D, A\} \cup \{S\} = \{C, D, A, S\} \\ \text{ANUL}_5 &= \{C, D, A, S\} \cup \emptyset = \{C, D, A, S\} \end{aligned}$$

$\therefore$ El conjunto de variables anulables es $\{C, D, A, S\}$. Al eliminar las producciones ε , tenemos:

$$\begin{aligned} S &\rightarrow AB \mid ACA \mid ab \mid B \mid CA \mid AA \mid AC \mid A \mid C \mid \varepsilon \\ A &\rightarrow aAa \mid B \mid CD \mid aa \mid C \mid D \\ B &\rightarrow bB \mid bA \mid b \\ C &\rightarrow cC \mid c \\ D &\rightarrow aDc \mid CC \mid ABb \mid ac \mid C \mid Bb \end{aligned}$$

6.3.3. Eliminación de producciones unitarias

Definición 6.9. (*Producción unitaria*) Una producción de la forma $A \rightarrow B$ donde A y B son variables, se llama *producción unitaria*.

Definición 6.10. (*Conjunto unitario*) El conjunto unitario de una variable A se define de la siguiente manera:

$$\text{UNIT}(A) := \{X \in V : \exists \text{ una derivación } A \Rightarrow^* X \text{ que usa únicamente producciones unitarias}\}$$

Por definición $A \in \text{UNIT}(A)$

Algoritmo para encontrar las producciones unitarias

El siguiente algoritmo sirve para encontrar todas las variables alcanzables de una GLC.

$\mathbf{UNIT}_1(A) := \{A\}$.

$\mathbf{UNIT}_{i+1} := \mathbf{UNIT}_i(A) \cup \{X \in V : \exists \text{ producción } Y \rightarrow X, Y \in (\mathbf{UNIT}_i(A))\}$.

De esta forma obtenemos una sucesión creciente de conjuntos de variables:

$$\mathbf{UNIT}_1 \subseteq \mathbf{UNIT}_2 \subseteq \mathbf{UNIT}_3 \subseteq \dots$$

El algoritmo se presenta entonces como:

- Inicializar: $\mathbf{UNIT}(A) := \{A\}$.
- Repetir: $\mathbf{UNIT}(A) := \mathbf{UNIT}(A) \cup \{X \in V : \exists \text{ producción } Y \rightarrow X, Y \in (\mathbf{UNIT}(A))\}$
- Hasta: No se añaden nuevas variables a **ANUL**.

Las producciones unitarias se pueden eliminar añadiendo para cada variable A las producciones (no unitarias) de las variables contenidas en el conjunto unitario $\mathbf{UNIT}(A)$.

Ejemplo 6.6. Eliminar las producciones unitarias de la siguiente gramática:

$$\begin{aligned} S &\rightarrow AS \mid AA \mid BA \mid \varepsilon \\ A &\rightarrow aA \mid a \\ B &\rightarrow bB \mid bC \mid C \\ C &\rightarrow aA \mid bA \mid B \mid ab \end{aligned}$$

Solución

$$\begin{aligned} \mathbf{UNIT}(S)_1 &= \{S\} \\ \mathbf{UNIT}(S)_2 &= \{S\} \cup \emptyset = \{S\} \\ \mathbf{UNIT}(A)_1 &= \{A\} \\ \mathbf{UNIT}(A)_2 &= \{A\} \cup \emptyset = \{A\} \\ \mathbf{UNIT}(B)_1 &= \{B\} \\ \mathbf{UNIT}(B)_2 &= \{B\} \cup \{C\} = \{B, C\} \\ \mathbf{UNIT}(B)_3 &= \{B, C\} \cup \{B\} = \{B, C\} \\ \mathbf{UNIT}(C)_1 &= \{C\} \\ \mathbf{UNIT}(C)_2 &= \{C\} \cup \{B\} = \{C, B\} \\ \mathbf{UNIT}(C)_3 &= \{C, B\} \cup \{C\} = \{C, B\} \end{aligned}$$

Eliminando las producciones unitarias se obtiene:

$$\begin{aligned} S &\rightarrow AS \mid AA \mid BA \mid \varepsilon \\ A &\rightarrow aA \mid a \\ B &\rightarrow bB \mid bC \mid aA \mid bA \mid ab \\ C &\rightarrow aA \mid bA \mid bB \mid bC \mid ab \end{aligned}$$

6.4. Forma Normal de Chomsky (FNC)

Una GLC está en *Forma Normal de Chomsky (FNC)* si cumple con:

1. No tiene variables inútiles.
2. No tiene producciones ε (exceptuando el posible caso de $S \rightarrow \varepsilon$)
3. Todas las producciones son de la forma $A \rightarrow a$ (producciones simples) o $A \rightarrow BC$ (producciones binarias).

En particular, una gramática FNC no tiene producciones unitarias.

Teorema 6.1. (*Procedimiento de conversión a FNC*) Toda GLC es equivalente a una gramática en Forma Normal de Chomsky.

Para transformar una GLC en FNC, seguiremos los algoritmos de la sección anterior en el siguiente orden:

1. Eliminar todas las variables no terminales.
2. Eliminar todas las variables no alcanzables.
3. Eliminar las producciones ε , exceptuando por supuesto el caso del símbolo inicial.
4. Eliminar las producciones unitarias.
5. Las producciones resultantes (que no sean $S \rightarrow \varepsilon$) son de la forma: $A \rightarrow a$ o $A \rightarrow w$, donde $|w| \geq 2$. Estas últimas se pueden simular con producciones de la forma $A \rightarrow BC$ o $A \rightarrow a$. Se introduce primero, para cada $a \in \Sigma$, una variable nueva T_a cuya única producción es $T_a \rightarrow a$. A continuación se introducen nuevas variables, con producciones binarias, para simular las producciones deseadas.

Dado que ya practicamos los pasos 1 al 4, ilustraremos un ejemplo del paso 5.

Ejemplo 6.7. Simular la producción $A \rightarrow abBaC$ con producciones simples y binarias.

Solución:

Introducimos las variables T_a y T_b y las producciones $T_a \rightarrow a$ y $T_b \rightarrow b$. Entonces $A \rightarrow abBaC$ se simula con:

$$\begin{aligned} A &\rightarrow T_a T_b B T_a C \\ T_a &\rightarrow a \\ T_b &\rightarrow b \end{aligned}$$

Ahora introducimos nuevas variables T_1, T_2, T_3 y las producciones binarias necesarias. Las únicas producciones de estas nuevas variables son las siguientes:

$$\begin{aligned}
A &\rightarrow T_a T_1 \\
T_1 &\rightarrow T_b T_2 \\
T_2 &\rightarrow B T_3 \\
T_3 &\rightarrow T_a C \\
T_a &\rightarrow a \\
T_b &\rightarrow b
\end{aligned}$$

Ejemplo 6.8. Encontrar una GLC en FNC equivalente a la siguiente gramática:

$$\begin{aligned}
S &\rightarrow AB \mid aBC \mid SBS \\
A &\rightarrow aA \mid C \\
B &\rightarrow bbB \mid b \\
C &\rightarrow cC \mid \varepsilon
\end{aligned}$$

Solución

El conjunto de variables terminales es:

$$TERM = \{B, C, S, A\}$$

y el conjunto de variables alcanzables es:

$$ALC = \{S, A, B, C\}$$

Es decir, la gramática no tiene variables inútiles. El conjunto de variables anulables es

$$ANUL = \{C, A\}$$

Al eliminar las producciones ε de G se obtiene la gramática:

$$\begin{aligned}
S &\rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\
A &\rightarrow aA \mid C \mid a \\
B &\rightarrow bbB \mid b \\
C &\rightarrow cC \mid c
\end{aligned}$$

A continuación encontramos los conjuntos unitarios de todas las variables:

$$\begin{aligned}
UNIT(S) &= \{S, B\} \\
UNIT(A) &= \{A, C\} \\
UNIT(B) &= \{B\} \\
UNIT(C) &= \{C\}
\end{aligned}$$

Al eliminar las producciones unitarias, tenemos:

$$\begin{aligned} S &\rightarrow AB \mid aBC \mid SBS \mid bbB \mid b \mid aB \\ A &\rightarrow aA \mid cC \mid c \mid a \\ B &\rightarrow bbB \mid b \\ C &\rightarrow cC \mid c \end{aligned}$$

Luego introducimos las variables nuevas T_a, T_b y T_c y las producciones $T_a \rightarrow a, T_b \rightarrow b$ y $T_c \rightarrow c$ con el propósito de que todas las producciones sean unitarias o de la forma $A \rightarrow w$ con $|w| \geq 2$.

$$\begin{aligned} S &\rightarrow AB \mid T_aBC \mid SBS \mid T_bT_bB \mid b \mid T_aB \\ A &\rightarrow T_aA \mid T_cC \mid c \mid a \\ B &\rightarrow T_bT_bB \mid b \\ C &\rightarrow T_cC \mid c \\ T_a &\rightarrow a \\ T_b &\rightarrow b \\ T_c &\rightarrow c \end{aligned}$$

Finalmente, se introducen nuevas variables, con producciones binarias, para simular las producciones de la forma $A \rightarrow w$ con $|w| \geq 2$.

$$\begin{aligned} S &\rightarrow AB \mid T_aT_1 \mid ST_2 \mid T_bT_3 \mid b \mid T_aB \\ A &\rightarrow T_aA \mid T_cC \mid c \mid a \\ B &\rightarrow T_bT_3 \mid b \\ C &\rightarrow T_cC \mid c \\ T_a &\rightarrow a \\ T_b &\rightarrow b \\ T_c &\rightarrow c \\ T_1 &\rightarrow BC \\ T_2 &\rightarrow BS \\ T_3 &\rightarrow T_bB \end{aligned}$$

Referencias

- [1] Favio E. Miranda, A. Liliana Reyes, et.al, *Notas de Clase para el curso de Autómatas y Lenguajes Formales*, Facultad de Ciencias, UNAM, Revisión 2019-2.
- [2] Rodrigo De Castro, *Notas de Clase de Teoría de la Computación: Lenguajes, autómatas, gramáticas*, Universidad Nacional de Colombia, Facultas de Ciencias, 2004.